

RELATÓRIO SOBRE O PROGRAMA DE APRESENTAÇÃO DE DADOS ABERTOS DO CONSELHO REGIONAL DE NUTRIÇÃO 1ª REGIÃO AO TCU

Versão sanitizada e documentada para handoff — credenciais e chaves de acesso removidas.

Nota de autoria e handoff

Este relatório e os códigos aqui documentados foram desenvolvidos por Adriana A. G. Silva durante seu estágio, com o objetivo de deixar o processo registrado e compreensível para a próxima pessoa que vier a dar continuidade a este trabalho.

As credenciais de acesso originalmente utilizadas nos testes foram removidas desta versão por segurança. Consulte a seção "Configuração de credenciais" para saber como obter e configurar novas credenciais antes de executar os códigos.

1. Introdução

Este relatório tem como objetivo descrever o programa desenvolvido para a apresentação dos dados abertos do Conselho Regional de Nutrição 1ª Região (CRN-1) ao Tribunal de Contas da União (TCU). O projeto visa garantir transparência e acessibilidade aos dados, permitindo a análise e auditoria das informações.

De acordo com o Acórdão nº 1648/2024 do TCU, os conselhos federais de fiscalização profissional foram determinados a elaborar e publicar, no prazo de 120 dias, um plano de dados abertos de forma integrada dentro de seus sistemas profissionais. Esse plano deve conter um cronograma detalhado com atividades, prazos, metas, responsáveis e indicadores. Ademais, as instâncias de auditoria interna de cada conselho devem avaliar e certificar o processo de publicação de dados abertos e transparência. O TCU também alertou que os conselhos que não atingirem 100% de transparência em dados abertos estão em desconformidade com a Lei de Acesso à Informação (Lei 12.527/2011).

2. Metodologia

Para a implementação do programa, seguimos as seguintes etapas:

- Levantamento dos requisitos e normas do TCU para a apresentação de dados.
- Coleta dos dados abertos disponibilizados pelo CRN-1.
- Desenvolvimento de um programa para tratamento, formatação e visualização das informações.
- Testes e validação da precisão dos dados apresentados.

3. Detalhes do Programa

O programa foi desenvolvido utilizando a linguagem de programação Python e bibliotecas especializadas para manipulação de dados, tais como Requests, OpenPyXL, JSON e BytesIO. Suas principais funcionalidades incluem:

- Extração de dados em formatos JSON e Excel a partir de fontes externas.

- Processamento e normalização dos dados para garantir consistência.
- Geração de relatórios padronizados para auditoria.
- Exportação dos dados formatados para arquivos compatíveis com as exigências do TCU.

4. Resultados e Conclusão

A implementação do programa permitiu a geração de relatórios padronizados, melhorando a transparência e conformidade dos dados do CRN-1 perante o TCU. Os testes realizados confirmaram a precisão dos dados apresentados, garantindo que as informações estão corretas e acessíveis.

Foram desenvolvidas soluções para um total de 14 questões, das quais 12 já foram concluídas. Neste momento, restam duas questões pendentes para a finalização do processo: a questão 5 e a questão 13. Essas questões ainda não foram resolvidas pois estavam aguardando a gerência fornecer as informações necessárias para sua conclusão.

Parte das questões foi obtida por meio do portal Implanta, seguindo o processo necessário para acesso ao código. As questões 1, 2, 3, 4, 9, 10, 11 e 12 foram geradas a partir desse portal. Todas as questões fornecidas foram devidamente testadas e verificadas. As questões que não foram disponibilizadas pelo Implanta ficaram inacessíveis devido à ausência das licenças necessárias.

Abaixo, são apresentados os programas desenvolvidos para cada questão, demonstrando a implementação detalhada das soluções adotadas.

Recomenda-se a manutenção periódica do programa e a revisão constante dos dados para assegurar a continuidade da conformidade com as normas do TCU.

4.1 Configuração de credenciais

Por questões de segurança, as credenciais de acesso à API do portal Implanta (chave e senha) não estão incluídas neste documento. Antes de executar qualquer um dos códigos das Questões 1 a 4 e 9 a 12, configure as variáveis de ambiente abaixo com as credenciais fornecidas pela plataforma Implanta:

```
# Linux / macOS
export IMPLANTA_CHAVE="sua_chave_aqui"
export IMPLANTA_SENHA="sua_senha_aqui"

# Windows (PowerShell)
$env:IMPLANTA_CHAVE="sua_chave_aqui"
$env:IMPLANTA_SENHA="sua_senha_aqui"
```

As credenciais podem ser solicitadas ao setor responsável pelo contrato com a plataforma Implanta no CRN-1.

5. Apresentação dos Códigos Desenvolvidos para Cada Questão

Esta seção contém os códigos-fonte desenvolvidos para resolver as questões identificadas durante a implementação do programa de apresentação de dados abertos do CRN-1 ao TCU. Cada código foi criado para atender a uma necessidade específica, conforme as exigências do Tribunal de Contas da União.

A seguir, os códigos são organizados por questão, com uma breve descrição de sua finalidade e como foram utilizados para processar, formatar e visualizar os dados. Os campos de chave e senha foram substituídos por variáveis de ambiente (ver seção 4.1).

Questão 1: Extração de Dados das Atas de Colegiados

O objetivo desta questão era realizar a extração das Atas de Colegiados disponíveis no portal Implanta, especificamente para o período de 01/01/2022 a 31/12/2023. A extração foi feita por meio de uma requisição HTTP para a API do portal, utilizando autenticação por variáveis de ambiente.

```
import http.client
import os

conn = http.client.HTTPSConnection("crn-df.implanta.net.br")
headers = {
    'Chave': os.environ["IMPLANTA_CHAVE"],
    'Senha': os.environ["IMPLANTA_SENHA"],
    'Accept': "application/json, text/json"
}

# Atas de colegiados no período informado
conn.request("GET", "/portaltransparencia/servico/api/AtasColegiados?
dataInicio=01%2F01%2F2022&dataTermino=31%2F12%2F2023", headers=headers)
res = conn.getresponse()
data = res.read()
print(data.decode("utf-8"))
```

Descrição do Código:

- O código realiza uma requisição GET à API do portal Implanta, solicitando as atas dos colegiados no período determinado.
- A autenticação é feita utilizando chave e senha lidas de variáveis de ambiente.
- A resposta da requisição é retornada no formato JSON e é impressa no terminal para verificação.

Questão 2: Extração de Dados dos Conselheiros

A tarefa desta questão envolvia a extração das informações dos conselheiros do CRN-1 a partir da API disponibilizada pelo portal Implanta.

```
import http.client
import os

conn = http.client.HTTPSConnection("crn-df.implanta.net.br")
headers = {
    'Chave': os.environ["IMPLANTA_CHAVE"],
    'Senha': os.environ["IMPLANTA_SENHA"],
    'Accept': "application/json, text/json"
}

# Dados dos conselheiros
conn.request("GET", "/portaltransparencia/servico/api/Conselheiros",
headers=headers)
res = conn.getresponse()
data = res.read()
```

```
print(data.decode("utf-8"))
```

Descrição do Código:

- O código faz uma requisição GET à API do portal Implanta para obter informações sobre os conselheiros.
- A autenticação é feita utilizando chave e senha lidas de variáveis de ambiente.
- A resposta da requisição é retornada no formato JSON e, em seguida, é impressa no terminal, permitindo verificar as informações recebidas.

Questão 3: Extração de Dados dos Planos de Cargos e Salários

O objetivo desta questão era realizar a extração dos dados relativos aos planos de cargos e salários do CRN-1, disponíveis no portal Implanta.

```
import http.client
import os

conn = http.client.HTTPSConnection("crn-df.implanta.net.br")
headers = {
    'Chave': os.environ["IMPLANTA_CHAVE"],
    'Senha': os.environ["IMPLANTA_SENHA"],
    'Accept': "application/json, text/json"
}

# Planos de cargos e salários
conn.request("GET", "/portaltransparencia/servico/api/PlanosCargosSalarios",
headers=headers)
res = conn.getresponse()
data = res.read()
print(data.decode("utf-8"))
```

Descrição do Código:

- O código realiza uma requisição GET à API do portal Implanta, solicitando os dados sobre os planos de cargos e salários.
- A autenticação é feita utilizando chave e senha lidas de variáveis de ambiente.
- A resposta da requisição é retornada no formato JSON e é impressa no terminal, permitindo a visualização e verificação das informações.

Questão 4: Extração de Dados do Quadro de Pessoas

Nesta questão, foi necessário realizar a extração dos dados referentes ao quadro de pessoas do CRN-1, disponíveis no portal Implanta.

```
import http.client
import os

conn = http.client.HTTPSConnection("crn-df.implanta.net.br")
headers = {
    'Chave': os.environ["IMPLANTA_CHAVE"],
    'Senha': os.environ["IMPLANTA_SENHA"],
    'Accept': "application/json, text/json"
```

```

}

# Quadro de pessoas
conn.request("GET", "/portaltransparencia/servico/api/QuadrosPessoas",
headers=headers)
res = conn.getresponse()
data = res.read()
print(data.decode("utf-8"))

```

Descrição do Código:

- O código faz uma requisição GET à API do portal Implanta para obter os dados relacionados ao quadro de pessoas do CRN-1.
- A autenticação é realizada utilizando chave e senha lidas de variáveis de ambiente.
- A resposta da requisição é retornada no formato JSON e é exibida no terminal para validação e verificação das informações recebidas.

Questão 5: Pendência

A questão 5 não foi resolvida durante o período do estágio, pois dependia da disponibilização das informações necessárias por parte da gerência. Assim que essas informações forem fornecidas, o processo de implementação e validação pode ser concluído de acordo com as exigências do TCU. Instruções de como completá-la estão na seção final deste documento.

Questão 6: Extração e Formatação dos Dados de Alienações de Bens Imóveis e Veículos

Nesta questão, foi necessário realizar a extração dos dados de alienações de bens imóveis e veículos do CRN-1, disponíveis em formato Excel no portal do CRN-1. O arquivo foi baixado e processado para o formato JSON.

```

import requests
import openpyxl
import json
from io import BytesIO

def get_relacao_de_alienacoes_bens_imoveis_veiculos():
    url =
    "https://novoportal.crn1.org.br/formularios/dados_abertos_crn1/relacao_de_alienacoes_bens_imoveis_veiculos.xlsx"

    response = requests.get(url)
    response.raise_for_status()

    wb = openpyxl.load_workbook(filename=BytesIO(response.content))
    sheet = wb.active
    data = []

    for row in sheet.iter_rows(values_only=True):
        data.append(row)

    headers = data[0]
    formatted_data = [dict(zip(headers, row)) for row in data[1:]]

```

```

        return formatted_data

tabela_json = get_relacao_de_alienacoes_bens_imoveis_veiculos()
json_string = json.dumps(tabela_json, indent=4, ensure_ascii=False)
print(json_string)

```

Descrição do Código:

- Requisição HTTP: o código baixa o arquivo Excel contendo os dados de alienações de bens imóveis e veículos.
- Processamento do Excel: o arquivo é carregado e lido com openpyxl, linha por linha.
- Formatação para JSON: os dados são organizados em lista de dicionários (cabeçalhos das colunas como chaves) e convertidos para JSON.
- Validação e Visualização: o JSON formatado é impresso no terminal para conferência.

Questão 7: Extração e Formatação do Relatório Discriminado de Passagens (histórico de contratos)

Nesta questão, foi necessário extrair os dados do relatório histórico de contratos do CRN-1, em formato Excel, tratando corretamente valores de data/hora na conversão para JSON.

```

import requests
import openpyxl
import json
from io import BytesIO
import datetime

def get_relatorio_discriminado_de_passagens():
    url =
    "https://novoportal.crn1.org.br/formularios/dados_abertos_crn1/relacao_historica_co
ntratos.xlsx"

    response = requests.get(url)
    response.raise_for_status()

    wb = openpyxl.load_workbook(filename=BytesIO(response.content))
    sheet = wb.active
    data = []

    for row in sheet.iter_rows(values_only=True):
        data.append(row)

    headers = data[0]
    formatted_data = [dict(zip(headers, row)) for row in data[1:]]

    return formatted_data

def datetime_converter(obj):
    if isinstance(obj, datetime.datetime):
        return obj.strftime('%Y-%m-%d %H:%M:%S')
    raise TypeError("Tipo não serializável")

tabela_json = get_relatorio_discriminado_de_passagens()
for item in tabela_json:
    for key, value in item.items():
        if isinstance(value, datetime.datetime):

```

```
        item[key] = value.strftime('%Y-%m-%d %H:%M:%S')
    json_string = json.dumps(tabela_json, indent=4, ensure_ascii=False)
    print(json_string)
```

Descrição do Código:

- Requisição HTTP: o código baixa o arquivo Excel contendo os dados do relatório.
- Processamento do Excel: os dados são extraídos linha por linha com openpyxl.
- Conversão para JSON: cada linha vira um dicionário, usando os cabeçalhos como chaves.
- Tratamento de Datas: valores datetime são convertidos para string no formato YYYY-MM-DD HH:MM:SS.
- Validação e Visualização: o JSON gerado é impresso no terminal para conferência.

Questão 8: Extração e Formatação da Relação de Transferências e Cooperações

Nesta questão, foi necessário extrair os dados de transferências e cooperações do CRN-1, disponíveis em formato Excel.

```
import requests
import openpyxl
import json
from io import BytesIO

def get_relacao_de_transferencias_cooperacoes():
    url = "https://novoportal.crn1.org.br/formularios/dados_abertos_crn1/relacao_de_transferencias_cooperacoes.xlsx"

    response = requests.get(url)
    response.raise_for_status()

    wb = openpyxl.load_workbook(filename=BytesIO(response.content))
    sheet = wb.active
    data = []

    for row in sheet.iter_rows(values_only=True):
        data.append(row)

    headers = data[0]
    formatted_data = [dict(zip(headers, row)) for row in data[1:]]

    return formatted_data

tabela_json = get_relacao_de_transferencias_cooperacoes()
json_string = json.dumps(tabela_json, indent=4, ensure_ascii=False)
print(json_string)
```

Descrição do Código:

- Requisição HTTP: o código baixa o arquivo Excel contendo os dados da relação de transferências e cooperações.
- Processamento do Excel: os dados são extraídos linha por linha com openpyxl.
- Formatação para JSON: os dados são convertidos para lista de dicionários e depois para JSON.

- Validação e Visualização: o JSON formatado é impresso no terminal para conferência.

Questão 9: Extração do Plano de Contas para o Exercício de 2022

Nesta questão, foi necessário extrair o plano de contas para o exercício de 2022, via API do portal Implanta.

```
import http.client
import os

conn = http.client.HTTPSConnection("crn-df.implanta.net.br")
headers = {
    'Chave': os.environ["IMPLANTA_CHAVE"],
    'Senha': os.environ["IMPLANTA_SENHA"],
    'Accept': "application/json, text/json"
}

# Plano de contas – exercício 2022
conn.request("GET", "/portaltransparencia/servico/api/PlanoDeContas?exercicio=2022", headers=headers)
res = conn.getresponse()
data = res.read()
print(data.decode("utf-8"))
```

Descrição do Código:

- Requisição HTTP: conexão HTTPS com o servidor do CRN-DF e requisição GET ao plano de contas de 2022.
- Headers de Autorização: chave e senha lidas de variáveis de ambiente, formato de resposta JSON.
- Processamento da Resposta: os dados são lidos e decodificados em UTF-8.
- Exibição dos Dados: os dados decodificados são impressos no terminal.

Questão 10: Extração do Balancete de Referência entre Janeiro de 2022 e Dezembro de 2023

Nesta questão, foi necessário extrair o balancete financeiro do CRN-DF, no período de janeiro de 2022 a dezembro de 2023.

```
import http.client
import os

conn = http.client.HTTPSConnection("crn-df.implanta.net.br")
headers = {
    'Chave': os.environ["IMPLANTA_CHAVE"],
    'Senha': os.environ["IMPLANTA_SENHA"],
    'Accept': "application/json, text/json"
}

# Balancete no período informado
conn.request("GET", "/portaltransparencia/servico/api/Balancete?referenciaInicio=01%2F2022&referenciaTermino=12%2F2023", headers=headers)
res = conn.getresponse()
data = res.read()
```



```
print(data.decode("utf-8"))
```

Descrição do Código:

- Requisição HTTP: conexão HTTPS e requisição GET ao balancete, com intervalo de datas.
- Headers de Autorização: chave e senha lidas de variáveis de ambiente, formato de resposta JSON.
- Processamento da Resposta: os dados são lidos e decodificados em UTF-8.
- Exibição dos Dados: os dados são exibidos no terminal para análise.

Questão 11: Extração da Execução Financeira de Janeiro de 2022

Nesta questão, foi necessário extrair os dados de execução financeira do CRN-DF para o mês de janeiro de 2022.

```
import http.client
import os

conn = http.client.HTTPSConnection("crn-df.implanta.net.br")
headers = {
    'Chave': os.environ["IMPLANTA_CHAVE"],
    'Senha': os.environ["IMPLANTA_SENHA"],
    'Accept': "application/json, text/json"
}

# Execução financeira – janeiro de 2022
conn.request("GET", "/portaltransparencia/servico/api/ExecucaoFinanceira?referenciaInicio=01%2F01%2F2022&referenciaTermino=31%2F01%2F2022", headers=headers)
res = conn.getresponse()
data = res.read()
print(data.decode("utf-8"))
```

Descrição do Código:

- Requisição HTTP: conexão HTTPS e requisição GET à execução financeira, especificando o período.
- Headers de Autorização: chave e senha lidas de variáveis de ambiente, formato de resposta JSON.
- Processamento da Resposta: os dados são lidos e decodificados em UTF-8.
- Exibição dos Dados: os dados são exibidos no terminal para análise.

Questão 12: Extração do Balanço Patrimonial de Referência entre Janeiro de 2022 e Dezembro de 2023

Nesta questão, foi necessário extrair o balanço patrimonial do CRN-DF, no período de janeiro de 2022 a dezembro de 2023.

```
import http.client
import os

conn = http.client.HTTPSConnection("crn-df.implanta.net.br")
headers = {
```

```

'Chave': os.environ["IMPLANTA_CHAVE"],
'Senha': os.environ["IMPLANTA_SENHA"],
'Accept': "application/json, text/json"
}

# Balanço patrimonial no período informado
conn.request("GET", "/portaltransparencia/servico/api/BalancoPatrimonial?
referenciaInicio=01%2F2022&referenciaTermino=12%2F2023", headers=headers)
res = conn.getresponse()
data = res.read()
print(data.decode("utf-8"))

```

Descrição do Código:

- Requisição HTTP: conexão HTTPS e requisição GET ao balanço patrimonial, com intervalo de datas.
- Headers de Autorização: chave e senha lidas de variáveis de ambiente, formato de resposta JSON.
- Processamento da Resposta: os dados são lidos e decodificados em UTF-8.
- Exibição dos Dados: os dados recebidos são exibidos no terminal para visualização.

Questão 13: Pendência

A Questão 13, assim como a Questão 5, não foi resolvida durante o período do estágio, por depender da disponibilização das informações necessárias pela gerência. As instruções para concluí-la estão na seção final deste documento.

Questão 14: Relatório Discriminado de Passagens

Nesta questão, foi necessário extrair os dados do "Relatório Discriminado de Passagens" do CRN-1, disponível em formato Excel, com tratamento de campos de data.

```

import requests
import openpyxl
import json
from io import BytesIO
import datetime

def get_relatorio_discriminado_de_passagens():
    url =
    "https://novoportal.crn1.org.br/formularios/dados_abertos_crn1/relatorio_discrimina
do_de_passagens.xlsx"

    response = requests.get(url)
    response.raise_for_status()

    wb = openpyxl.load_workbook(filename=BytesIO(response.content))
    sheet = wb.active
    data = []

    for row in sheet.iter_rows(values_only=True):
        data.append(row)

    headers = data[0]
    formatted_data = [dict(zip(headers, row)) for row in data[1:]]

```

```

        return formatted_data

def datetime_converter(obj):
    if isinstance(obj, datetime.datetime):
        return obj.strftime('%Y-%m-%d %H:%M:%S')
    raise TypeError("Tipo não serializável")

tabela_json = get_relatorio_discriminado_de_passagens()
for item in tabela_json:
    for key, value in item.items():
        if isinstance(value, datetime.datetime):
            item[key] = value.strftime('%Y-%m-%d %H:%M:%S')
json_string = json.dumps(tabela_json, indent=4, ensure_ascii=False)
print(json_string)

```

Descrição do Código:

- Requisição e Download do Arquivo: o código baixa o arquivo Excel do CRN-1.
- Leitura do Arquivo Excel: os dados da planilha são extraídos linha por linha com openpyxl.
- Conversão para JSON: a primeira linha vira os cabeçalhos, usados para transformar cada linha subsequente em um dicionário.
- Conversão de Datas: valores datetime são convertidos para o formato YYYY-MM-DD HH:MM:SS com strftime.
- Geração do JSON: a lista de dicionários é convertida para JSON, com indentação de 4 espaços, e impressa no terminal.

6. Como Completar as Questões 5 e 13

Para as questões 5 e 13, é necessário fazer o upload de cada tabela no site do Implanta, copiar o link gerado e colá-lo na variável url = "COLAR LINK" do script correspondente, garantindo que o link esteja entre aspas. O restante da lógica (download, leitura da planilha, conversão para JSON) segue o mesmo padrão usado nas Questões 6, 7, 8 e 14.

7. Salvar e Rodar o Código

Após configurar as variáveis de ambiente (seção 4.1) e, se necessário, colar o link na variável url, siga as instruções abaixo para salvar e rodar o código no Visual Studio Code:

7.1 Salvar o Código

- Abra o Visual Studio Code e cole o código na janela de edição.
- Clique em Arquivo > Salvar ou use o atalho Ctrl + S (Windows).

7.2 Rodar o Código

- No Visual Studio Code, com o código aberto, clique no ícone de executar no canto superior direito da janela (um ícone de triângulo verde).

- O código será executado automaticamente e o relatório gerado aparecerá no terminal integrado, abaixo da área de edição.

Sobre este documento

Documento originalmente elaborado por Adriana A. G. Silva durante seu estágio no CRN-1, organizado para servir de referência à próxima pessoa responsável por este processo. Versão sanitizada (sem credenciais) preparada para compartilhamento público em portfólio.